문자 수준 CNN과 DistilBERT 기반 캐스케이드 웹 공격 탐지의 속도–정확도 교환 분석

Speed–Accuracy Trade-off Analysis of Cascaded Character-Level CNN and DistilBERT for Web Attack Detection

요 약

인라인 웹 애플리케이션 방화벽(web application firewall, WAF)에서 다중 웹 공격을 분류할 때 문자 수준 합성곱 신경망(character-level convolutional neural network, char-CNN)은 처리 속도가 빠르다. 그러나 수용 영역이 짧아 세미콜론이나 파이프 같은 구분자를 공유하는 SQL 삽입(SQL injection, SQLi)과 명령 삽입(command injection, CmdI)을 자주 혼동한다. 반대로 사전학습 언어 모델인 DistilBERT는 긴 문맥을 읽어 이 오분류를 줄이지만, 단일 스트림 CPU에서 요청당 수십 밀리초가 걸려 실시간 경로에 단독으로 두기 어렵다. 본 연구는 대다수 요청을 char-CNN으로 즉시 처리하고 확신도가 낮은 요청만 DistilBERT로 넘기는 2단계 캐스케이드를 구성하여, SR-BH 2020 데이터셋과 단일 스레드 CPU(배치 1) 환경에서 실사용자 정상 트래픽 기준으로 평가하였다. 실험에서 char-CNN → DistilBERT 캐스케이드는 시험 요청의 0.7~0.8%만 2차로 넘겼다. 그 결과 char-CNN 단독 대비 전체 Macro-F1(0.9884에서 0.9911로 +0.28%p, 확인 분할 +0.53%p)과 CmdI F1(0.9682에서 0.9776으로 +0.94%p, 확인 분할 +1.74%p)이 올랐고, SQLi를 CmdI로 잘못 분류하던 오류가 절반 이상 줄어 보안 관제 시 유효한 CmdI 경고를 더 확보할 수 있었다. 추론 속도 역시 DistilBERT 단독보다 합산 12.3배(확인 분할 10.6배) 빨랐다. 다만 분할 seed에 따라 2차 단독 모델 대비 상대 손실폭이 달라졌고, 예측 클래스별 임계값 조정은 추가 분할에서 재현되지 않았으며, URL 인코딩 변형 시 1차 탐지 회피와 지연 증가가 함께 관찰되었다.

ABSTRACT

In inline web application firewalls (WAFs), character-level convolutional neural networks (char-CNNs) classify HTTP requests quickly. Because their receptive fields are short, however, they often confuse SQL injection (SQLi) and command injection (CmdI) payloads that share delimiters such as semicolons and pipes. Pre-trained transformers such as DistilBERT reduce this confusion using wider context, yet their single-stream CPU latency is too high for direct inline use. We pair char-CNN as a fast first stage with fine-tuned DistilBERT as a second stage, deferring only low-confidence requests. We test this cascade and an internal early-exit variant on the SR-BH 2020 dataset under single-thread CPU (batch 1) latency and browser-based benign traffic. Deferring 0.7–0.8% of requests, the char-CNN to DistilBERT cascade raises overall Macro-F1 by +0.28 pp (0.9884 to 0.9911; +0.53 pp on the confirmation split) and CmdI F1 by +0.94 pp (0.9682 to 0.9776; +1.74 pp on the confirmation split) over stand-alone char-CNN. It removes more than half of SQLi-to-CmdI false alerts, which lets security analysts issue more reliable CmdI warnings, while running 12.3× faster (10.6× on the confirmation split) than stand-alone DistilBERT. Even so, relative accuracy loss against stand-alone DistilBERT varies across fine-tuning splits, class-wise thresholds do not generalize to new splits, and URL-encoded payloads cause both latency growth and first-stage evasion.

Keywords: Web attack detection, Cascade inference, Character-level CNN, DistilBERT, Command injection, Inference latency

I. 서론

웹 애플리케이션 방화벽(web application firewall, WAF)은 서버 앞단 인라인 경로에서 모든 HTTP 요청을 검사한다. 따라서 탐지 정확도 못지않게 요청 한 건당 처리 지연이 중요하다. 한편 보안 기계학습 분야에서는 실험실 벤치마크 조건이 실제 운영 트래픽과 달라 성능이 과대평가되는 문제가 꾸준히 보고되었다[1].

본 연구는 별도의 수동 특징 사전(n-gram 빈도 등) 없이 원시 바이트나 서브워드 토큰을 직접 입력받는 딥러닝 웹 공격 분류기를 다룬다. 다중 공격 분류를 보안 관제(security operations center, SOC)에 적용할 때 가장 까다로운 지점은 정상 대 공격의 단순 이진 차단보다 SQL 삽입(SQL injection, SQLi)과 명령 삽입(command injection, CmdI)의 구분이다. 두 공격은

세미콜론(;), 파이프(|), 따옴표(') 등 국소 구분자를 똑같이 사용한다. 특히 운영체제 명령을 실행하는 CmdI는 서버 장악으로 이어지는 치명적 공격이지만, 실제 트래픽에서 차지하는 비율은 매우 낮다(본 데이터셋 기준 전체의 2.1%, SQLi의 약 1/33). 이 때문에 다수 클래스인 SQLi의 일부가 CmdI로 잘못 분류되거나 긴 CmdI 요청이 입력 길이 제한에 잘려 누락되면, 관제 화면의 CmdI 경보 정밀도가 크게 떨어진다. 문자 수준 합성곱 신경망(character-level convolutional neural network, char-CNN)[2]은 단일 스레드 CPU에서 0.35~0.60ms 만에 추론을 마치지만 짧은 합성곱 커널만으로는 SQLi와 CmdI의 혼동을 다 줄이지 못한다. 반대로 사전학습 트랜스포머인 DistilBERT[3]는 긴 문맥을 참조해 이 혼동을 줄여 주나, 요청당 45~46ms가 소요되어 단독 인라인 배치가 어렵다.

두 모델의 장단점을 절충하는 대표적인 방법이 가벼운 1차 모델로 대부분의 요청을 먼저 처리하고, 확신도가 임계값 τ보다 낮은 요청만 무거운 2차 모델로 넘기는 캐스케이드(cascade)[4] 및 조기 종료(early exit)[5] 구조이다. 하지만 웹 공격 탐지에서 이러한 구조를 실제 운영 기준에 맞춰 검증하려면 몇 가지 조건을 동시에 따져보아야 한다. 토큰화와 바이트 인코딩까지 포함한 단일 스트림(배치 1) 지연에서도 char-CNN 대비 F1 향상과 DistilBERT 대비 큰 폭의 가속이 함께 유지되는지 확인해야 한다. 또한 공격 표본이 과반수인 시험 집합이 아니라 낮은 실제 공격 기저율(π = 3.3%)[6]과 일반 브라우저 정상 트래픽 분포에서도 넘김 비율이 낮게 억제되는지, 그리고 단일 임계값이나 예측 클래스별 임계값[7]이 분할 seed가 바뀌어도 일관되게 작동하는지 검토할 필요가 있다. 공격자가 의도적으로 2차 호출을 유도하는 지연 유발 공격(slowdown attack)[8]에 대한 취약성도 확인 대상이다.

본 논문은 실트래픽 기반 다중 공격 데이터셋인 SR-BH 2020[9] 위에서 위 조건들을 실험으로 점검하였다. 주요 내용은 다음과 같다.

- **char-CNN**과 **DistilBERT** 결합 캐스케이드의 속도–정확도 교환 분석. 동일한 5-fold 분할과 단일 스레드 CPU 위에서 초경량 1차 캐스케이드(char-CNN-tiny → char-CNN), 내부 조기 종료(char-CNN-EE), 주 캐스케이드(char-CNN → DistilBERT), 조기 종료 출구 결합 캐스케이드(k3 exit → DistilBERT)를 비교하였다. 비교 기준선으로는 선행 연구(WAMM[10])의 DistilBERT를 동일 환경에서 재구현해 사용하였다. 주 캐스케이드는 char-CNN 단독 대비 전체 Macro-F1을 +0.28~+0.53%p(4개 분할 전체 +0.06~+0.53%p) 높이면서 DistilBERT 단독 대비 합산 **12.3**배(확인 분할 **10.6**배; 조기 종료 출구를 1차로 결합 시 **23.8~29.3**배) 빠르게 동작하였다. 아울러 2차 모델 미세조정 편차 때문에 사전 정확도 허용폭(2차 단독 대비 손실 0.30%p 이내)이 기본 분할(−0.03%p)에서는 통과하고 확인 분할(−0.32%p)에서는 0.02%p 차로 벗어나는 경계를 그대로 보고한다.
- **SQLi–CmdI** 오분류 감소를 통한 **CmdI** 경보 신뢰도 개선 및 **128** 토큰 잘림 분석. char-CNN → DistilBERT 캐스케이드는 시험 요청의 0.7~0.8%만 넘기면서도 char-CNN이 틀리던 SQLi→CmdI 오분류 감소분의 56~74%를 회수하고 실제 CmdI 미탐(CmdI→Normal)도 줄였다. 이로써 CmdI F1이 0.9682에서 0.9776(확인 분할 0.9618에서 0.9792)으로, CmdI 경보 정밀도가 95.2%에서 97.1%(확인 분할 93.9%에서 97.3%)로 높아져 관제 환경에서 유효한 CmdI 경고를 더 낼 수 있음을 확인하였다. 반면 선행 연구의 128 토큰 제한은 긴 CmdI 요청의 26.5%를 잘라내어 char-CNN 단독보다도 CmdI 탐지 성능이 낮아졌다.
- 실사용자 정상 트래픽 분리 평가와 임계값 조정·변형 공격의 한계 확인. 단일 임계값에서 공격 분류 점수를 맞추려 임계값을 높이면, 1차가 정상으로 맞게 예측하고도 확신도가 낮았던 실사용자 정상 요청이 대거 넘어가는 현상(특정 폴드 r_N = 33.6%)을 확인하였다. 이를 줄이려던 예측 클래스별 임계값은 추가 분할(seed 44·45) 실험에서 재현되지 않았고, 공격 페이로드에 URL 인코딩을 1회 적용했을 때 공격 요청당 기대 지연이 **6.4**배 늘어나며 1차 탐지 회피가 일어나는 문제도 함께 측정하였다.

실험 결과를 본 뒤 판정 기준을 바꾸는 일(HARKing)을 막기 위해, 모든 확인 실험의 통과 조건은 측정 전 저장소 커밋(2a456d7~d9d3678)에 미리 기록해 두었다. 사전 기준 밖의 관찰은 사후 분석 또는 병기로 구분해 기술한다.

II. 관련 연구

HTTP 요청을 문자열이나 바이트 시퀀스로 보고 딥러닝에 입력하는 접근은 웹 공격 탐지에서 널리 쓰인다[11]. 그중 Zhang 등[2]이 제안한 char-CNN은 별도의 어휘 사전 없이 원시 바이트에서 국소 n-gram

특징을 뽑아내는 경량 모델이다. 데이터셋 측면에서 Sureda Riera 등[9]의 SR-BH 2020은 실운영 웹 서버 앞단의 ModSecurity 로그를 수동 검수해 CAPEC 공격 라벨을 붙인 다중 클래스 벤치마크이다. 이 데이터 위에서 Osama 등[10]은 중복 제거를 거친 WAMM 프레임워크로 DistilBERT[3] 등의 분류 성능을 측정하였고, 이후 이중 채널 BERT 구조인 WADBERT[12]와 라벨 정제 기반 이상 탐지 기법인 X-WAD[13]가 발표되었다. 본 연구는 Osama 등[10]에서 가장 높은 딥러닝 점수를 낸 DistilBERT를 동일 분할과 동일 CPU 위에서 512 토큰 및 128 토큰 설정으로 재구현하여 비교 기준으로 삼는다.

캐스케이드 추론은 1차 모델의 최대 소프트맥스 확률(maximum softmax probability, MSP)[4][14]을 임계값과 비교해 2차 호출 여부를 결정한다. Tasdemir 등[6]은 SQLi 이진 탐지에서 경량 1차와 BERT 2차를 묶어 약 20배의 가속을 보고하였다. 넘김(deferral) 규칙 설계는 선택적 분류[15] 및 전문가 넘김 학습[16] 연구와 연결된다. 다만 Enomoto와 Eda[17]는 온도 보정[18]이 오히려 캐스케이드 효율을 해칠 수 있음을 지적했고, Jitkrittum 등[7]은 최적 넘김 조건이 1·2차 정답 확률의 차이에 좌우되므로 2차가 특정 클래스에만 강할 때 1차 확신도만으로는 한계가 있음을 보였다. 단일 신경망 중간 층에 보조 분류기를 다는 조기 종료(BranchyNet)[5]는 두 모델을 따로 돌릴 때 생기는 전처리 중복을 줄여 준다. 반면 입력을 조작해 조기 종료를 방해하는 지연 유발 공격(DeepSloth)[8]이 컴퓨터 비전 분야에서 보고된 바 있으며, 인라인 보안 장비의 성능을 논할 때는 단일 스트림 지연 측정(MLPerf)[19]과 낮은 실제 공격 기저율(base rate)[20]을 함께 고려해야 한다.

III. 방법

3.1 입력 표현과 딥러닝 캐스케이드 구성

각 HTTP 요청에서 URI와 본문(body)을 이어 붙인 원시 바이트열을 $b = (b_0, b_1, ..., b_{L-1})$로 표기한다. **char-CNN**[2] 계열 모델은 고정 길이 패딩을 두지 않고 상한 2,304바이트 안에서 실제 길이 L 그대로의 1차원 바이트 시퀀스를 입력받는다. 본 데이터셋의 URI+본문 길이는 중앙값 96바이트, 99백분위수 352바이트, 최댓값 674바이트여서 상한에 걸려 잘리는 요청은 없다(가장 큰 커널 폭인 7바이트보다 짧은 입력만 7바이트까지 0으로 채운다). 각 바이트(257개 기호)를 임베딩 벡터로 바꾼 뒤 1차원 합성곱과 시간축 최대 풀링을 거쳐 선형 분류기로 4개 클래스를 예측한다. 2차 참조 모델인 **DistilBERT**[3]는 원문 문자열을 소문자화 기반 WordPiece 토크나이저로 쪼개어 입력받는다.

본 연구는 빠른 기준선인 **char-CNN**(임베딩 64차원, 커널 폭 3·5·7 각 128채널 병렬, 141,252 파라미터)과 무거운 참조 모델인 **DistilBERT**(`distilbert-base-uncased`, 66.96M 파라미터) 사이에서 네 가지 딥러닝 구성을 실험한다 (Table 1).

1. 초경량 **1**차 캐스케이드(`char-CNN-tiny` → `char-CNN`): 임베딩을 32차원으로 줄이고 커널 폭 3짜리 분기 하나(64채널, 14,692 파라미터)만 둔 초경량 모델을 1차에, 기본 char-CNN을 2차에 배치한 구성이다.
2. 내부 조기 종료(`char-CNN-EE`): 별도 1차 모델을 두는 대신 기본 char-CNN의 커널 3 분기 풀링 출력(128차원) 뒤에 보조 선형 분류기(128→4)를 덧붙인 형태이다(141,768 파라미터). 학습할 때는 주 분류기 손실과 보조 분류기 손실을 $L = L_{main} + 0.3\ L_{aux}$로 합산해 함께 학습한다. 추론할 때는 임베딩과 커널 3 분기만 먼저 실행해 보조 분류기 확신도가 임계값 이상이면 바로 결과를 내고, 임계값보다 낮을 때만 나머지 커널 5·7 분기를 마저 계산한다. 보조 출구와 똑같은 구조(임베딩 64, 커널 3 단일 분기, 128채널, 41,668 파라미터)를 주 분류기 없이 단독 학습한 대조군은 **char-CNN-k3**로 부른다.
3. 주 캐스케이드(`char-CNN` → `DistilBERT`): 기본 char-CNN을 1차로 삼고 미세조정된 DistilBERT를 2차로 연결한 2단계 캐스케이드이다. 선행 연구(WAMM[10])는 최대 토큰 길이를 128로 제한하였으나, 본 데이터셋의 학습+검증 풀(226,048행)에서 128 토큰 상한을 걸면 전체의 6.9%(15,553행), 특히 페이로드가 긴 CodeInjection의 24.7%와 CmdI의 26.5%에서 뒷부분이 잘려 나간다. 이에 본 실험에서는 문맥 잘림이 없는 **512** 토큰(토큰 수 중앙값 54, 99백분위수 189)을 기본 2차 모델로 사용하고, 선행 연구와 같은 **128** 토큰 설정(`DistilBERT-128`)은 5.3절에서 따로 비교한다.
4. 조기 종료 출구 결합 캐스케이드(`k3 exit` → `DistilBERT`): char-CNN-EE에서 결합 학습된 커널 3 보조 출구만 떼어 1차에 두고,

2차에는 512 토큰 DistilBERT를 연결해 1차 연산 시간까지 줄인 구성이다.

Table 1. Deep-learning models compared in 5-fold cross-validation

Model	Role & setup	Input	Params
char-CNN	Baseline ref.; 2nd stage of tiny cascade; 1st stage of main cascade	raw bytes (L)	141,252
char-CNN-tiny	1st stage of tiny cascade (k3, 64-ch)	raw bytes (L)	14,692
char-CNN-k3	Single-branch stand-alone control (k3, 128-ch)	raw bytes (L)	41,668
char-CNN-EE	Joint-trained internal early exit; 1st stage of k3-exit cascade	raw bytes (L)	141,768
DistilBERT	High-accuracy ref. (WAMM[10]); 2nd stage of DistilBERT cascades	≤512 (or 128) tok.	66.96M

Notes: All models are trained with class-weighted cross-entropy on the same 5 folds (early stopping patience = 5 on internal validation Macro-F1; DistilBERT fine-tuned for 3 epochs using Adam, lr = 2e-5, batch size = 16).

3.2 넘김 판단 규칙과 배포 기저율 재가중 합산 배수

1차 모델 h_1이 입력 x에 대해 낸 클래스별 소프트맥스 확률을 $p_1(k|x)$, 최대 소프트맥스 확률을 $s(x) = \max_k p_1(k|x)$라 하자. 임계값 τ를 쓸 때 2단계 캐스케이드의 최종 예측 $\hat{y}(x)$는 식 (1)과 같다.

$$\hat{y}(x) = h_1(x) \text{ if } s(x) \geq \tau, \text{ otherwise } h_2(x) \quad (1)$$

기본 단일 임계값**(single threshold)** 규칙은 각 폴드의 내부 검증 집합에서 캐스케이드의 검증 Macro-F1이 2차 단독 검증 Macro-F1 대비 0.5%p 이내를 유지하는 후보 중 배포 기대 넘김률(식 3)이 가장 작은 τ 하나를 골라 시험 폴드에 그대로 적용한다. 이와 비교하는 예측 클래스별 임계값 **(class-wise threshold)** 규칙은 클래스마다 확신도 분포가 다른 점을 고려해, 1차가 예측한 클래스 $\hat{c}_1(x) = \text{argmax}_k p_1(k|x)$마다 별도의 임계값 τ_c를 둔다.

$$\text{defer } x \Leftrightarrow s(x) < \tau_{\hat{c}_1(x)}, \ \hat{c}_1(x) = \text{argmax}_k p_1(k|x) \quad (2)$$

네 클래스의 임계값 조합 $(\tau_1, ..., \tau_4)$은 내부 검증 집합에서 전체 넘김 예산을 0.5%p씩 늘려 가며 단위 넘김당 검증 Macro-F1 상승량이 가장 큰 클래스의 임계값을 먼저 높이는 탐욕적(greedy) 탐색으로 구한다.

1차 모델의 요청당 지연을 t_1, 2차 모델 지연을 t_2, 비교 기준 단독 모델 지연을 t_{ref}라 둔다(char-CNN-EE는 앞단 커널 3 출구 지연 t_{front}를 t_1으로, 뒷단 커널 5·7 지연 t_{rest}를 t_2로, 기본 char-CNN 지연을 t_{ref}로 두며, DistilBERT 결합 캐스케이드는 DistilBERT 지연 t_R을 t_{ref}로 둔다). 정상 요청 넘김률(r_N)과 공격 요청 넘김률(r_A)이 서로 다르므로, 실제 운영 환경의 공격 기저율 π **= 3.3%**[6]를 대입한 기대 넘김률 E_π와 폴드 f의 기대 지연 t_f는 식 (3)으로 계산한다.

$$E_\pi = \pi \cdot r_A + (1 - \pi) \cdot r_N, \ t_f = t_1 + E_\pi \cdot t_2 \quad (3)$$

5개 교차검증 폴드에서 구한 폴드별 속도 배수 $S_f = t_{ref} / t_f$를 단순히 산술평균하면, 넘김이 많이 발생한 특정 폴드의 지연 비용이 희석되어 전체 배수가 부풀려진다. 이를 막기 위해 본 논문은 다섯 폴드의 평균 지연 비용을 기준으로 산출한 합산 속도 배수 $\bar{S}$**(**폴드별 배수 S_f의 조화평균**)**를 주 지표로 삼고, 산술평균은 괄호 안에 함께 적는다.

$$\bar{S} = t_{ref} / \left((1/5) \Sigma_f t_f \right) \quad (4)$$

IV. 실험 환경

4.1 데이터셋과 실사용자 정상 층 분리

실험 데이터는 SR-BH 2020[9] 원본 907,815행의 CAPEC 다중 라벨을 정상(Normal), SQL 삽입(SQLi), 코드 삽입(CodeInjection, 서버 측 PHP 코드 실행 탐침 위주), 명령 삽입(CmdI)의 4개 클래스로 묶어 구성하였다. 정상 라벨에 섞여 있던 공격 의심 구문(정규표현식 감사 적발 30,280건), 동일 문자열의 라벨 충돌 행, 완전 중복 행을 걸러내어 총 **265,939**행(학습 186,157 · 검증 39,891 · 봉인 시험 39,891; 공격 비중 74.8%)을 남겼다. 입력 필드는 URI와 본문만 쓴다(쿠키 헤더를 덧붙인 입력은 쿠키 값을 가린 대조군과 비교했을 때 보정 p = 0.17로 유의한 차이가 없어 제외하였다). 가장 작은 클래스인 CmdI는 전체 265,939행 중 **5,551**건**(2.1%)**, 5-fold 교차검증에 쓰이는 학습+검증 풀(226,048행) 기준 **4,718**건**(**학습 **3,886**건· 검증 **832**건**, 2.1%)**으로 다수 공격인 SQLi(학습+검증 풀 156,039건, 69.0%)의 약 1/33 크기이다.

전처리 후 데이터를 점검하는 과정에서 정상(Normal) 트래픽의 심한 출처 편향이 확인되었다. 학습+검증 풀의 정상 요청 56,906건(전체 66,948건)을 원본 로그의 User-Agent별로 나누어 보니, 자동 취약점 스캐너(Nikto, OpenVAS 등)에서만 나온 요청이 **96.0%(54,600건;** 전체 **64,250건)**였고 스캐너와 일반 브라우저 양쪽에서 관찰된 요청이 **3.1%(1,756건;** 전체 **2,053건)**였다. 즉 스캐너에서 유래한 정상 텍스트가 99.0%(단일 최빈 스캐너 User-Agent 행 기준 97.8%)를 차지하며, 스캐너 없이 일반 브라우저 User-Agent로만 수집된 순수 실사용자 정상 요청은 학습+검증 풀 전체에 **550건(1.0%,** 폴드당 약 **110건;** 전체 데이터셋 **645건)**뿐이었다. 실제 5-fold 교차검증에서도 char-CNN의 실사용자 정상 오탐률(false positive rate, FPR)은 **2.00%(seed 42; seed 43**은 **1.27%)**로 나와 스캐너 정상 오탐률**(0.35%**, seed 43은 0.24%)보다 **5.7**배**(seed 43**은 **5.3**배**)** 높았다. char-CNN → DistilBERT 캐스케이드의 정상 넘김률 r_N 역시 스캐너가 대부분인 검증 분포에서는 0.86%(seed 43은 0.79%)에 그쳤지만, 실사용자 정상 요청에서는 평균 **7.08%(**약 **8.2**배**,** 특정 폴드 최대 **33.6%; seed 43**은 평균 **8.48%**로 약 **10.7**배**)**로 크게 뛰었다. 스캐너 위주의 전체 정상 지표를 그대로 쓰면 오탐과 넘김 비용이 실제보다 훨씬 작게 보이므로, 본 연구는 모든 평가에서 실사용자 정상 층과 스캐너 정상 층을 분리하고 실사용자 정상 $\mathbf{r_N}$을 주 판정 기준으로 삼는다.

4.2 평가 절차와 사전 고정 판정 기준

클래스 간 표본 수 차이가 크므로 분류 성능은 4개 클래스 F1의 단순 평균인 **Macro-F1**으로 평가한다. 모든 모델 학습과 캐스케이드 평가는 봉인 시험 집합(39,891행)을 빼둔 학습+검증 풀 **226,048**행 위에서 동일한 층화 5-fold 교차검증으로 진행하였다(각 폴드의 학습 데이터 중 10%를 내부 검증 집합으로 떼어 조기 종료와 임계값 τ 선택에 사용). 폴드별 점수 차이는 학습 데이터 중복을 반영한 Nadeau–Bengio 보정 대응 t-검정[21]과 Holm 다중비교 보정으로 검정하였다. 기본 분할 seed는 42로 두고, 재현성 확인에는 seed 43(클래스별 임계값 추가 확인은 seed 44·45)을 사용하였다.

추론 지연은 단일 데스크톱(AMD Ryzen 7 9800X3D, RAM 32GB, RTX 4080 SUPER, Windows 11, PyTorch 2.6.0)에서 인라인 WAF의 요청 건별 처리를 모사해 **CPU** 단일 스레드**(1 thread)**·배치 **1**로 측정하였다. 원문 문자열이 들어와 바이트 인코딩이나 토큰화를 거치고 신경망 순전파를 마쳐 예측값이 나올 때까지의 종단간 시간을 검증 표본 2,000건에 대해 재고(20회 워밍업 후 30회 반복), 그 중앙값(p50)을 대표 지연으로 썼다[19]. 연속 실행 사이의 미세한 클럭 변동(4~7%)과 측정 일자별 OS 배경 부하 차이(예: 서로 다른 측정 시점 간 char-CNN 중앙값 0.345ms 대 0.563~0.604ms)가 배수 계산에 섞이지 않도록, 비교할 모델들은 항상 한 실행 안에서 요청마다 번갈아 재고 같은 실행의 측정값끼리만 나누어 배수를 구하였다.

비교 기준 단독 모델 대비 우위("위")로 판정하기 위한 사전 기준은 측정 전에 세 가지로 고정하였다(실사용자 정상, $\pi = 3.3\%$ 기준): ① 합산 속도 배수 $\hat{\mathbf{S}} \geq \mathbf{1.2}$배**,** ② **5**개 폴드 모두 $\mathbf{S_f > 1.0}$배**,** ③ 기준 모델 대비 **Macro-F1** 평균 손실 $\leq \mathbf{0.30\%p}(\Delta F1 \geq -0.30\%p)$. 세 조건을 동시에 만족할 때만 기준 모델보다 위에 있다고 판정한다.

V. 실험 결과

5.1 딥러닝 캐스케이드의 속도–정확도 교환: char-CNN 대비 F1 개선과 DistilBERT 대비 12배 가속

본 학습에 앞서 기본 char-CNN의 구간별 실행 시간을 먼저 측정한 결과, 전체 추론 시간의 **91.0%**가 세 합성곱 분기**(**커널 **3·5·7)**에서 쓰였고 바이트 인코딩과 파이썬 호출 고정비는 **8.5%**에 그쳤다. 즉 커널 수를 줄이거나 앞 분기에서 일찍 나가는 구조로 실제 지연 단축이 가능함을 확인하였다. 2차 후보로 재구현한 512 토큰 DistilBERT 단독 모델은 5-fold 교차검증에서 Macro-F1 **0.9914**(seed 42)와 **0.9947**(seed 43)을 기록해 char-CNN 단독**(0.9884** 및 **0.9862**)보다 높았다. Table 2와 Fig. 1에 네 가지 딥러닝 구성의 측정 결과를 정리하였다.

Table 2. Speed–accuracy trade-off of deep-learning cascades vs. stand-alone references (5-fold CV, CPU batch 1, 1 thread, end-to-end unpadded latency, real-user benign traffic at $\pi = 3.3\%$)

Configuration	Reference R (F1)	Seed	Threshold rule	Macro-F1 (ΔR / Δchar-CNN)	r_N / r_A	Pooled $\bar{S}$ (arith.*)	Fold S_f range	Verdict vs. R
char-CNN-tiny → char-CNN	char-CNN (0.9884)	42	Single τ	0.9839 (−0.45 pp / −0.45 pp)	30.2% / 4.0%	2.13× (2.39×*)	1.52–4.10×	No (F1 loss)
char-CNN-EE (internal early exit)	char-CNN (0.9884)	42	Single τ	0.9874 (−0.10 pp / −0.10 pp)	1.1% / 0.4%	**3.52×** (3.53×*)	3.24–3.62×	**Yes**
char-CNN-EE (confirm.)	char-CNN (0.9862)	43	Single τ	0.9895 (+0.33 pp / +0.33 pp)	0.0% / 0.005%	**3.62×** (3.62×*)	3.62–3.62×	**Yes (2/2)**
Control: char-CNN-k3 (solo)	char-CNN (0.9884)	42	—	0.9748 (−1.36 pp / −1.36 pp)	0.0% / 0.0%	3.62×	3.62–3.62×	No (F1 loss)
char-CNN solo vs. DistilBERT	DistilBERT (0.9914)	42	—	0.9884 (−0.303 pp / 0.00 pp)	0.0% / 0.0%	80.2×	80.2–80.2×	No (0.003 pp short)
char-CNN → DistilBERT	DistilBERT (0.9914)	42	Single τ	**0.9911 (−0.03 pp / +0.28 pp)**	7.1% / 0.7%	**12.3×** (51.6×*)	2.95–80.2×	**Yes (seed 42)**
char-CNN → DistilBERT (confirm.)	DistilBERT (0.9947)	43	Single τ	**0.9914 (−0.32 pp / +0.53 pp)**	8.5% / 0.7%	**10.6×** (38.3×*)	3.93–80.2×	No (p = 0.038 vs. R)
char-CNN → DistilBERT (class-wise τ)	DistilBERT (0.9914)	42	Class-wise τ	0.9920 (+0.06 pp / +0.37 pp)	0.7% / 0.7%	51.1× (57.5×*)	32.5–80.2×	Secondary (failed on s44–45)
char-CNN → DistilBERT (class-wise τ)	DistilBERT (0.9947)	43	Class-wise τ	0.9920 (−0.27 pp / +0.58 pp)	1.1% / 0.6%	42.5× (50.6×*)	13.7–80.2×	Secondary (failed on s44–45)
k3 exit → DistilBERT	Main cascade (12.2×)	42	Single τ	0.9883 (−0.28 pp / −0.00 pp)	3.2% / 0.4%	**29.3×** (86.8×*, ×2.40)	15.0–289.8×	**Yes (vs. main cascade)**
k3 exit → DistilBERT (confirm.)	Main cascade (10.5×)	43	Single τ	0.9919 (+0.05 pp / +0.57 pp)	4.0% / 0.4%	**23.8×** (124.9×*, ×2.27)	12.1–289.8×	**Yes (2/2 vs. main cascade)**

Notes: Values with * are arithmetic means of fold speedups. Interleaved p50 latencies per run: char-CNN cascade run gives char-CNN-tiny $t_1 = 0.0606$ ms, char-CNN $t_2 = 0.3451$ ms, k3 front $t_{front} = 0.0954$ ms; main DistilBERT cascade run (seeds 42–43) gives char-CNN $t_2 = 0.563$ ms, DistilBERT(512) $t_R = 45.15$ ms (80.2×); token-comparison and k3-exit run (seeds 44–45 and k3-exit cascade) gives $t_2 = 0.604$ ms, $t_{front} = 0.159$ ms, $t_R(512) = 46.10$ ms, $t_R(128) = 46.08$ ms.

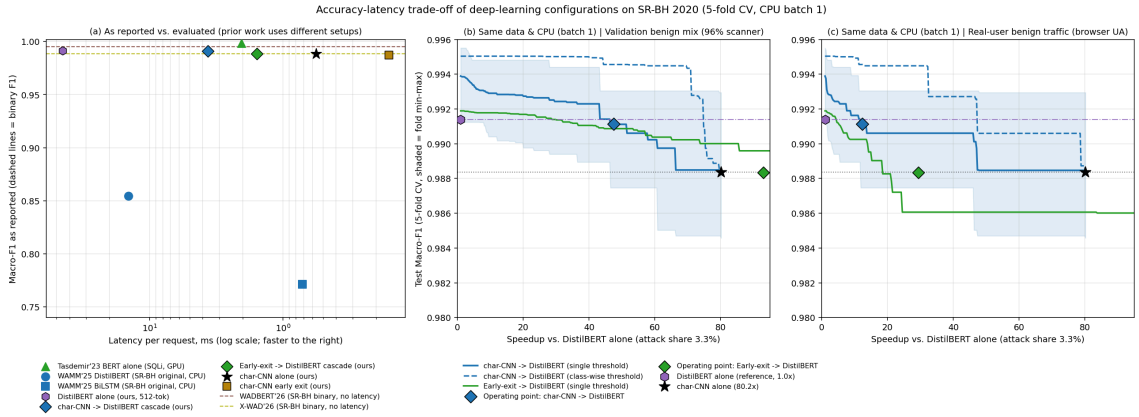


Fig. 1. Accuracy–latency comparison of deep-learning configurations on SR-BH 2020 (5-fold CV, CPU batch 1, 1 thread). (a) Per-request latency and F1 alongside prior studies under their reported setups; (b) and (c) 5-fold Macro-F1 versus speedup over stand-alone DistilBERT on the same folds under validation benign mix (96% scanner) and real-user benign traffic.

1) 초경량 **1**차 캐스케이드(char-CNN-tiny → char-CNN)**: 1**차 표현력 부족**.** 1차 모델(t_1 = 0.0606ms)의 단독 Macro-F1이 0.9601에 머물러 실사용자 정상 요청의 30.2%가 2차로 넘어갔다. 합산 배수는 **2.13**배(산술 2.39배)였고, 캐스케이드 Macro-F1은 0.9839로 기본 char-CNN보다 **0.45%p** 낮아 허용 기준(손실 0.30%p 이내)을 넘지 못했다.

2) 내부 조기 종료(char-CNN-EE)**:** 넘김이 아닌 결합 학습 보조 출구의 압축 효과**.** 커널 3 분기에 보조 분류기를 붙여 함께 학습한 char-CNN-EE는 seed 42에서 합산 **3.52**배(산술 3.53배, F1 0.9874, 단독 대비 −0.10%p), 확인 분할인 seed 43에서 합산 **3.62**배(F1 0.9895, 단독 대비 +0.33%p)를 기록해 두 분할 모두 사전 기준을 통과하였다. 다만 폴드별 내역을 뜯어보면 seed 42의 5개 폴드 중 3개, seed 43의 5개 폴드 전부가 뒷단 넘김률 **0%**였다(보조 출구 단독 F1이 이미 0.9849·0.9894). 즉 속도 이득은 어려운 요청을 골라 넘긴 덕분이 아니라 보조 출구 자체가 충분히 정확했기 때문이다. 같은 크기의 신경망을 주 분류기 없이 혼자 학습한 char-CNN-k3(seed 42, F1 **0.9748**, 기본 char-CNN 대비 −1.36%p)는 결합 학습된 보조 출구보다 1.01%p 낮아 기준에 미달하였다(보정 p = 0.34; 최고 검증 점수가 나온 에폭이 보조 출구는 폴드별 8·10·2·19·3이었고 k3 단독은 3·20·9·6·2였다). 조기 종료 인내 횟수(patience)를 늘린 별도 실험은 하지 않았으므로, 이 1.01%p 차이가 큰 모델과 함께 학습한 덕분인지 단독 학습의 이른 종료 탓인지는 단정할 수 없다. 따라서 본 결과는 넘김 게이트의 효과라기보다 결합 학습을 통한 단일 분기 모델 압축으로 볼 수 있다.

3) 주 캐스케이드(char-CNN → DistilBERT)**: char-CNN** 대비 **F1** 향상과 **DistilBERT** 대비 **12**배 가속**.** 같은 실행 안에서 잰 char-CNN(0.563ms)과 512 토큰 DistilBERT(45.15ms)의 처리 시간 차이는 **80.2**배이다. char-CNN을 단독으로 쓰면 80.2배 빠르지만 DistilBERT보다 Macro-F1이 seed 42에서 0.303%p(0.9884 대 0.9914), seed 43에서 0.85%p(0.9862 대 0.9947) 낮고, 특히 표본이 적은 CmdI 클래스에서 오답이 많이 나온다. 반면 두 모델을 이은 char-CNN → DistilBERT 캐스케이드(단일 임계값)는 공격 요청의 0.7%, 실사용자 정상 요청의 7.1~8.5%(전체 시험 집합의 0.7~0.8%)만 2차로 넘기면서 양쪽 기준선의 약점을 보완하였다. 먼저 **1**차인 **char-CNN** 단독과 비교하면 전체 **Macro-F1**이 **seed 42**에서 **+0.28%p(0.9884 → 0.9911), seed 43**에서 **+0.53%p(0.9862 → 0.9914)** 올랐다**.** 5.2절의 seed 44(+0.07%p)·seed 45(+0.06%p)까지 포함하면 4개 분할 모두에서 char-CNN 단독보다 높은 점수를 냈다(4클래스 평균에 대한 5-fold 보정 t-검정에서는 p = 0.34·0.23으로, 점수 상승이 소수 클래스인 CmdI F1 [+0.94%p, +1.74%p]에 집중되어 있기 때문이며 그 상세 내역은 5.3절에서 다룬다). 동시에 **2**차인 **DistilBERT** 단독과 비교하면 실사용자 정상 기준 합산 배수로 **seed 42**에서 **12.3**배(산술 **51.6**배, 요청당 평균 **3.67ms), seed 43**에서 **10.6**배(산술 **38.3**배, 평균 **4.28ms)** 빨라져 45ms대 지연을 4ms 안팎으로 줄였다. 다만 DistilBERT 단독 대비 상대 손실폭은 **seed 42**에서 **−0.03%p(0.9911** 대 **0.9914), seed 43**에서 **−0.32%p(0.9914** 대 **0.9947,** 보정 **p = 0.038)**로 나타났다. 사전 고정 기준(손실 0.30%p 이내)에 비추면 **seed 42**에서는 통과했으나, 확인용 seed

43에서는 0.02%p 차이로 기준을 넘지 못했다. 수치 변화를 보면 캐스케이드 자체의 Macro-F1은 0.9911에서 0.9914로 거의 같았지만, 비교 대상인 DistilBERT 단독 점수가 분할 및 3 에폭 미세조정 차이로 인해 0.9914에서 0.9947로 올라간 것이 원인이었다. 캐스케이드는 실사용자 정상 기준 약 93%의 트래픽을 1차 char-CNN에서 끝내므로, 2차 단독 점수가 더 올라가도 1차가 높은 확률로 틀린 일부 표본은 교정되지 않는다.

4) 조기 종료 출구 결합 캐스케이드**(**k3 exit → DistilBERT**): 1**차 경량화로 **24~29**배 가속**.** 주 캐스케이드의 1차를 기본 char-CNN(0.604ms)에서 결합 학습된 k3 보조 출구(0.159ms)로 교체하고 2차는 동일한 DistilBERT(46.10ms)로 두었을 때, DistilBERT 단독 대비 합산 배수는 **seed 42**에서 **29.3**배**(**주 캐스케이드 **12.2**배 대비 **2.40**배 가속**, F1 0.9883,** 주 캐스케이드 대비 **−0.28%p)**, **seed 43**에서 **23.8**배**(**주 캐스케이드 **10.5**배 대비 **2.27**배 가속**, F1 0.9919,** 주 캐스케이드 대비 **+0.05%p)**로 높아졌다. 두 분할 모두 주 캐스케이드 대비 사전 기준을 충족하였다. 다만 seed 42의 −0.28%p는 허용 경계(−0.30%p)에 가깝고, 가장 빠른 폴드(289.8배)는 넘김률이 0%여서 가속의 주된 원인은 1차 실행 시간이 약 1/4로 줄어든 데(0.159ms 대 0.604ms) 있다.

5.2 넘김 판단의 한계: 느린 폴드 진단과 예측 클래스별 임계값 재확인

단일 임계값에서 특정 폴드가 느려지는 이유**(**사후 진단**).** seed 42의 char-CNN → DistilBERT에서 합산 배수(12.3배)가 산술평균(51.6배)과 크게 벌어진 까닭은 폴드 4 하나의 배수가 **2.95**배(나머지 네 폴드는 46.9~80.2배)로 낮았기 때문이다. 폴드 4의 내부 검증 집합에서는 검증 Macro-F1 목표치를 맞추기 위해 넘김 예산이 3.0%(임계값 $\tau = 0.985$)까지 올라갔는데, 이 임계값을 시험 폴드에 적용하자 실사용자 정상 요청의 **33.6%**(Wilson 95% 신뢰구간 [25.4%, 43.0%], 107건 중 36건; 내부 검증에서도 28.9%)가 2차로 넘어갔다. 이때 넘어간 실사용자 정상 36건 가운데 **34**건은 **1**차 **char-CNN**이 이미 **Normal**로 맞게 분류했지만 최대 확률이 0.985보다 조금 낮았던 요청이었다. 즉 단일 임계값 방식에서는 공격 클래스 간 오분류를 줄이려고 임계값을 높이면, 원래 맞게 분류되던 실사용자 정상 트래픽까지 대거 2차로 넘어가는 비용이 발생한다.

예측 클래스별 임계값의 추가 분할**(seed 44·45)** 재현 실패**.** 위 현상 때문에 seed 42와 43에서 예측 클래스별 임계값을 시험했을 때는 1차가 Normal로 예측한 요청의 임계값이 낮게 잡혀 정상 넘김이 줄었고, 합산 배수가 51.1배·42.5배로 크게 올랐다 (Table 2). 이 결과가 다른 분할에서도 그대로 재현되는지 확인하고자 새로운 분할 **seed 44**와 **45**에서 사전 고정 조건(① 단일 임계값 대비 합산 배수 20% 이상 증가, ② Macro-F1 손실 0.30%p 이내, ③ 폴드별 기대 넘김률 E_π 차이 보정 $p < 0.05$)으로 실험한 결과, 두 분할 모두 통과 조건을 다 채우지 못했다.

- **seed 44**: 단일 임계값 합산 배수 **76.0**배 → 클래스별 임계값 **59.6**배**(−21.6%,** 조건 ① 미달**)**, Macro-F1 0.9908 → 0.9920(char-CNN 단독 0.9901, DistilBERT 단독 0.9901; 조건 ② 충족), E_π 차이 보정 $p = $ **0.34(**조건 ③ 미달**)**.
- **seed 45**: 단일 임계값 합산 배수 **42.4**배 → 클래스별 임계값 **54.4**배**(+28.4%,** 조건 ① 충족**)**, Macro-F1 0.9903 → 0.9913(char-CNN 단독 0.9897, DistilBERT 단독 0.9927; 조건 ② 충족), E_π 차이 보정 $p = $ **0.54(**조건 ③ 미달**)**.

폴드별 내역을 보면 클래스별 임계값이 속도를 높인 경우는 단일 임계값이 실사용자 정상을 많이 넘기는 높은 τ를 고른 폴드(seed 45의 폴드 1: 17.6배에서 32.5배로 상승)에 한정되었다. 반대로 seed 44에서는 단일 임계값이 다섯 폴드 모두에서 이미 상한 근처(합산 76.0배, $r_N = 0.0\%$, $r_A = 0.1\%$)를 골랐기 때문에 줄일 정상 넘김이 없었고, 클래스별 임계값이 오히려 두 폴드에서 넘김을 늘려 배수를 깎았다. 결국 seed 42·43에서 보였던 큰 배수 상승은 새 분할에서 일관되게 재현되지 않았으며(배수 조건 1/2, 유의성 0/2), 임계값을 클래스별로 나누는 방법만으로는 안정적인 가속을 얻기 어렵다. (참고로 이 추가 실험에서 char-CNN → DistilBERT 단일 임계값은 같은 seed의 DistilBERT 단독과 비교해 seed 44에서 +0.08%p, seed 45에서 −0.24%p를 기록해 4개 분할[seed 42~45] 중 3개에서 손실 0.30%p 이내에 들었다.)

5.3 SQLi–Cmdl 오분류 개선을 통한 유효한 Cmdl 경고 확보와 128 토큰 문맥 잘림 검증

char-CNN 뒤에 80배 무거운 DistilBERT를 붙였을 때 구체적으로 어떤 오류가 고쳐지는지 확인하기 위해, 5-fold 전체 예측값(226,048행)의 혼동행렬을 분해하고 선행 연구(WAMM[10])의 128 토큰 설정 재구현 결과를 함께 비교하였다(Table 3).

Table 3. Pooled 5-fold error decomposition (226,048 rows; True SQLi = 156,039, True CmdI = 4,718) comparing stand-alone char-CNN, the proposed char-CNN → DistilBERT cascade (single threshold), stand-alone DistilBERT (512 tokens), and the 128-token DistilBERT setup

Metric / error count	s42 char-CNN	s42 Cascade (ours)	s42 DistilBERT(512)	s42 DistilBERT(128)	s43 char-CNN	s43 Cascade (ours)	s43 DistilBERT(512)	s43 DistilBERT(128)
Overall Macro-F1	0.9884	**0.9911**	0.9914	0.9718	0.9862	**0.9914**	0.9947	0.9777
Missed attack (Attack → Normal)	791	**697 (−11.9%)**	736 (−7.0%)	773	839	**743 (−11.4%)**	777 (−7.4%)	826
└ Missed CmdI (CmdI → Normal)	28	**25**	23	31	30	**25**	23	67
Benign false alarm (Normal → Attack)	206	247	245	253	140	164	118	194
Real-user / scanner FPR	2.00% / 0.35%	2.00% / 0.42%	1.82% / 0.43%	2.00% / 0.43%	1.27% / 0.24%	1.27% / 0.28%	1.09% / 0.21%	1.27% / 0.33%
Attack-type confusion (total)	**266**	**196 (−26.3%)**	**167 (−37.2%)**	**934**	**359**	**220 (−38.7%)**	**59 (−83.6%)**	**629**
└ SQLi → CmdI false alerts	**192**	**123 (−35.9%)**	**99 (−48.4%)**	**847**	**252**	**113 (−55.2%)**	**2 (−99.2%)**	**551**
CmdI Precision / Recall (%)	95.18 / 98.52	**97.07 / 98.45**	97.26 / 98.41	83.52 / 98.47	93.90 / 98.58	**97.32 / 98.52**	99.64 / 98.43	88.24 / 97.84
CommandInjection (CmdI) F1	**0.9682**	**0.9776 (+0.94 pp)**	**0.9784 (+1.02 pp)**	**0.9038**	**0.9618**	**0.9792 (+1.74 pp)**	**0.9903 (+2.85 pp)**	**0.9284**

1) SQLi–CmdI 오분류 감소와 보안 관제에서의 유효한 **CmdI** 경고 확보**.** Table 3을 보면 공격을 정상으로 잘못 보낸 전체 미탐(Attack → Normal)은 char-CNN 단독에서 512 토큰 DistilBERT 단독으로 바꿀 때 두 분할 모두 약 **7%**(791→736건, 839→777건) 줄었고, char-CNN → DistilBERT 캐스케이드에서는 **697**건**(−11.9%, seed 42)**·**743**건**(−11.4%, seed 43)**으로 단독 모델보다도 적었다. 실제 CmdI를 정상으로 놓친 건수(CmdI → Normal) 역시 char-CNN 단독의 28건·30건에서 캐스케이드 **25**건·**25**건으로 줄어들었다(정답으로 잡은 CmdI는 4,718건 중 4,645건·4,647건). 실사용자 정상 오탐률(n = 550)은 2.00% 대 1.82%(seed 42), 1.27% 대 1.09%(seed 43)로 요청 1건 차이였다. 오류 감소가 가장 뚜렷한 부분은 **SQLi**와 **CmdI** 사이의 공격 종류 혼동이다. 학습+검증 풀(226,048행)에서 SQLi(156,039건, 69.0%)는 CmdI(4,718건, 2.1%)보다 33배 많다. 이 때문에 char-CNN이 세미콜론(;), 파이프(|), 따옴표(') 같은 짧은 기호만 보고 SQLi나 정상 요청 일부를 CmdI로 잘못 분류하면(seed 42에서 SQLi→CmdI 192건·Normal→CmdI 38건, seed 43에서 SQLi→CmdI 252건·Normal→CmdI 50건), 표본이 적은 CmdI 경보 목록에 거짓 경보가 쌓여 **CmdI** 정밀도**(Precision)**가 **95.18% (seed 42)**·**93.90%(seed 43)**로 낮아진다(반면 실제 CmdI가 SQLi로 분류된 건수는 char-CNN 46건·36건, 캐스케이드 48건·45건으로 전체 CmdI의 1% 미만이다). 전체 시험 요청 가운데 1차 char-CNN의 최대 확률이 0.99 이상인 비율은 97.0%였으나, char-CNN이 틀리고 DistilBERT가 맞힌 표본들에서는 char-CNN 확신도 중앙값이 **0.775(seed 42)**·**0.766(seed 43)**로 낮았고 0.99 이상인 비율도 **15.1%**·**10.7%**에 그쳤다. 즉 char-CNN이 SQLi와 CmdI 사이에서 흔들릴 때는 확신도 자체가 낮게 형성된다. 그 덕분에 char-CNN → DistilBERT 캐스케이드는 시험 요청의 **0.7~0.8%**만 **2**차로 넘기면서도 **SQLi→CmdI** 오분류를 **seed 42**에서 **192→123**건**(DistilBERT** 단독 **99**건까지 감소량의 **74%** 회수**), seed 43**에서 **252→113**건**(**단독 **2**건까지 감소량의 **56%** 회수**)**으로 줄였고**, Normal→CmdI** 오탐도 절반**(38→17**건**, 50→23**건**)**으로 낮추었다**.** 이에 따라 캐스케이드의 **CmdI** 경보 정밀도는 **97.07% (+1.89%p, seed 42)**·**97.32%(+3.42%p, seed 43)**로**, CmdI F1**은 **0.9682 → 0.9776(+0.94%p, seed 42)** 및 **0.9618 → 0.9792(+1.74%p, seed 43)**로 올랐다(seed 44·45에서도 CmdI F1 +0.29%p·+0.26%p 상승). 운영체제 명령 실행을 노리는 CmdI 경보는 관제

인력이 우선 확인해야 하는데, 캐스케이드를 통해 SQLi발 오탐을 걸러냄으로써 실제 대응이 필요한 **CmdI** 경고를 더 높은 신뢰도로 낼 수 있다.

2) 선행 연구 설정**(128** 토큰**)** 재구현과 긴 **CmdI** 문맥 잘림 확인. 선행 연구(WAMM[10])처럼 입력 상한을 128 토큰으로 줄여 학습한 DistilBERT는 512 토큰 모델보다 Macro-F1이 크게 떨어졌고(seed 42 **0.9718** 대 **0.9914**로 **−1.96%p**, seed 43 **0.9777** 대 **0.9947**로 **−1.70%p**, 둘 다 보정 $p = 0.006$), 80배 가벼운 char-CNN 단독(0.9884·0.9862)보다도 낮았다. 점수 하락은 128 토큰에서 뒷부분이 잘린 15,553행(6.9%)에 몰려 있었다(잘린 행 정답률: seed 42 0.994→0.957, seed 43 0.996→0.956; 잘리지 않은 행은 0.1%p 이내). 특히 요청 뒷부분에 셸 명령이 길게 붙는 CmdI는 26.5%가 잘려 나가면서, seed 43에서 CmdI 미탐(CmdI→Normal)이 23건에서 67건으로 늘어나 CmdI 정답 건수가 4,646건에서 4,616건으로 줄었고, 뒷부분이 잘린 SQLi를 CmdI로 오인하는 건수도 99→847건(seed 42)·2→551건(seed 43)으로 급증해 **CmdI F1**이 **0.9038·0.9284**까지 떨어졌다(Table 3). 한편 배치 1 추론에서는 고정 패딩 없이 실제 토큰 수만큼만 연산하므로, 전체의 93.1%가 128 토큰 이하인 본 데이터셋에서는 중앙값 지연이 512 토큰과 거의 같았다(46.10ms 대 46.08ms). `char-CNN → DistilBERT-128` 캐스케이드는 DistilBERT-128 단독보다 합산 **67.5**배**(F1 +1.66%p, seed 42)·76.3**배**(+0.84%p, seed 43)** 빨라 수치상 우위 기준을 넘겼으나, 내부 검증에서 선택된 넘김률이 사실상 0%였다. 즉 이는 캐스케이드의 효과라기보다 바이트 전체를 다 읽는 **char-CNN** 단독이 **128** 토큰에서 잘린 **DistilBERT**보다 **CmdI** 분류에 훨씬 유리하다는 점을 보여준다.

3) 템플릿 묶음 분할 및 다중 **seed** 편차 점검. 시험 폴드 요청의 66.5%(CmdI는 14.7%)는 학습 폴드에 숫자와 긴 토큰만 다른 동일 골격 템플릿을 가지고 있다. 같은 템플릿이 학습과 시험에 나뉘어 들어가지 못하도록 묶음 분할(group split)로 다시 평가했을 때도 char-CNN의 Macro-F1은 0.9894(+0.10%p)로 유지되어 템플릿 암기에 따른 점수 부풀림(사전 기각선 −2.0%p)은 나타나지 않았다. 또한 4개 분할 seed(42~45)에서 char-CNN 단독 점수의 표준편차는 0.18%p(0.9862~0.9901)로 작았으나, 3 에폭 미세조정한 DistilBERT 단독 점수는 0.9901~0.9947로 움직여 사전 허용폭(0.30%p)과 맞먹는 분산을 보였다.

VI. 논의 및 한계

운용 목적에 따른 모델 선택. 실험 결과는 인라인 WAF 및 관제 환경의 요구사항에 따라 서로 다른 구성을 선택해야 함을 보여준다. 1. 정상 대 공격의 이진 차단이 주 목적이거나 약 **0.30~0.85%p**의 **Macro-F1** 차이를 감수할 수 있는 경우, 2차 트랜스포머 없이 기본 **char-CNN** 단독 또는 결합 학습된 내부 조기 종료 모델**(**`char-CNN-EE`**, k3** 보조 출구**)**만 두는 편이 낫다. 512 토큰 DistilBERT보다 **80.2**배**(char-CNN** 단독**, F1** 차이 **−0.30~−0.85%p)**에서 **284~289**배**(**`char-CNN-EE`**, F1** 차이 **−0.40~−0.52%p)** 빠르면서 전체 공격 미탐 건수는 큰 차이가 없다. 2. 관제 화면에서 고위험 **CmdI** 경보를 **SQLi**와 정확히 갈라내어 우선 처리해야 하는 경우, `char-CNN → DistilBERT` 캐스케이드**(**합산 **10.6~12.3**배 가속**,** 평균 **3.7~4.3ms)**나 1차를 보조 출구로 바꾼 `k3 exit → DistilBERT` 캐스케이드**(**합산 **23.8~29.3**배 가속**,** 평균 **1.6~1.9ms)**가 적합하다. 전체 트래픽의 0.7~0.8%만 2차로 보내면서도 char-CNN 단독 대비 전체 Macro-F1(+0.28~+0.53%p)과 CmdI F1(+0.94~+1.74%p), CmdI 정밀도(93.9~95.2% → 97.1~97.3%)를 높여, 단일 스레드 CPU에서도 수 밀리초 안의 평균 지연으로 신뢰도 높은 CmdI 경보를 얻을 수 있다.

연구의 한계. 본 실험 결과를 해석할 때는 몇 가지 제약 조건을 염두에 두어야 한다. 우선 모든 추론 시간은 데스크톱 CPU 1대의 단일 스레드·배치 1·PyTorch 기본 실행 환경에서만 측정하였다. 다중 스레드 CPU나 GPU 배치 서빙 환경에서는 트랜스포머 연산이 병렬화되어 1·2차 지연 격차(t_R / t_1)가 줄어들고, 그만큼 캐스케이드의 상대 배수도 작아질 수 있다. 또한 데이터셋 내 순수 실사용자 정상 요청이 학습+검증 풀 기준 550건(폴드당 약 110건)뿐이어서 실사용자 정상 넘김률 r_N의 폴드 간 변동이 크다(5.2절에서 특정 폴드 배수가 2.95배로 떨어진 직접적 원인).

선행 연구(WAMM[10]) 재구현에서는 원 설정인 128 토큰 대신 문맥 잘림을 없앤 512 토큰과 Adam 최적화기를 기본 비교선으로 삼았으며, 128 토큰 조건은 5.3절에서 별도로 대조하였다. 한편 1차가 확률 0.99 이상으로 틀리고 2차가 맞힌 오답 110건(대조군 110건)의 원문을 확인했을 때 70.9%(대조군 31.8%)에서 원본 데이터의 라벨 오류가 의심되었으나, 이는 대형 언어모델 단독 검토 결과여서 복수의 보안 전문가 교차 감사를 거치지

않은 참고 수치로만 남긴다. 평가는 단일 공개 데이터셋(SR-BH 2020)에서 수행되었으므로 타 웹 로그로의 일반화는 추가 검증이 필요하다.

끝으로 비용 기반 지연 유발 공격(slowdown attack) [8]에 대한 취약성을 확인하기 위해, 가중치 저장용으로 다시 학습한 seed 42 체크포인트(비변형 기준 단일 임계값 합산 23.5배, 클래스별 임계값 45.7배) 위에서 공격 페이로드에 고정 변형을 가해 보았다. 모델 가중치나 구조 정보 없이 공격 문자열에 URL 인코딩 1회(`url_encode`)만 적용해도 1차 char-CNN의 공격 넘김률 r_A가 단일 임계값 기준 0.27%에서 8.76%로 약 **32**배 뛰었고, 공격 요청 한 건당 기대 지연은 0.73ms에서 4.64ms로 **6.4**배 늘었다(4종 변형 평균 공격 넘김률 배율은 단일 임계값 ×11.0, 클래스별 임계값 ×2.78). 이는 이미지 분류에서 보고된 DeepSloth[8] 형태의 지연 유발 공격이 웹 페이로드의 단순 인코딩 변형만으로도 일어날 수 있으며, 2차가 1차보다 약 76배 무거워 넘김 증가가 곧바로 지연 상승으로 이어진다는 점을 보여준다. 물론 변형된 공격 요청의 기대 지연(4.64ms)도 DistilBERT 단독(46.1ms)보다는 약 10배 짧고, 공격 기저율 3.3% 환경에서 전체 트래픽 평균 지연 증가분은 약 0.13ms 수준이다. 오히려 더 주의해야 할 대목은 `url_encode` 적용 시 **1**차 **char-CNN**이 공격으로 잡아내는 비율 자체가 **99.4%**에서 **86.7%**로 떨어졌다는 점이다. 1차가 높은 확률로 정상이라 잘못 판단한 변형 공격은 2차로 넘어가지조차 않아 그대로 탐지를 빠져나가므로, 지연 증가보다 **1**차 탐지 회피**(evasion)**가 더 직접적인 위협이 된다(이를 막으려면 입력 앞단에 반복 URL 디코딩 정규화를 두거나 변형 페이로드를 학습에 포함해야 한다).

VII. 결론

본 논문은 인라인 WAF 환경을 가정하여 문자 수준 CNN(char-CNN)과 사전학습 트랜스포머(DistilBERT)를 이은 2단계 캐스케이드 및 내부 조기 종료 구조의 속도–정확도 교환을 SR-BH 2020 데이터셋에서 평가하였다. 실험에서 `char-CNN → DistilBERT` 캐스케이드는 시험 요청의 0.7~0.8%만 2차로 넘기면서 char-CNN 단독 대비 전체 Macro-F1(+0.28~+0.53%p)과 CmdI F1(+0.94~+1.74%p)을 높였고, SQLi를 CmdI로 잘못 분류하던 오류의 56~74%를 제거해 보안 관제에서 신뢰할 수 있는 CmdI 경고를 더 확보하였다. 동시에 단일 스레드 CPU 종단간 지연 기준 DistilBERT 단독보다 합산 **12.3**배(확인 분할 **10.6**배; 결합 학습된 보조 출구를 1차에 둔 구성은 **23.8~29.3**배) 빠른 추론 속도를 보였다. 아울러 선행 연구의 128 토큰 입력 제한이 긴 CmdI 구문을 잘라내어 큰 점수 하락(−1.70~−1.96%p)을 일으킨다는 점도 확인하였다. 반면 분할 seed에 따른 2차 단독 모델 대비 상대 손실 변동(−0.03%p 대 −0.32%p), 단일 임계값의 실사용자 정상 과잉 넘김(특정 폴드 33.6%)과 클래스별 임계값의 재현 실패, 그리고 URL 인코딩 변형 시 나타나는 공격 지연 상승(**6.4**배) 및 1차 탐지 회피(99.4% → 86.7%)도 함께 드러났다. 이러한 결과는 웹 공격 탐지 캐스케이드에서 임계값 규칙 조정보다 1·2차 모델 간 지연 격차와 1차 모델의 입력 정규화 강건성이 더 중요하며, 캐스케이드를 평가할 때는 전처리를 포함한 종단간 지연, 실제 공격 기저율, 실사용자 정상 넘김률, 다중 분할 편차를 함께 점검해야 함을 시사한다.

References

[1] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," Proceedings of the 31st USENIX Security Symposium, pp. 3971-3988, Aug. 2022.

[2] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," Advances in Neural Information Processing Systems (NIPS), pp. 649-657, Dec. 2015.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, Oct. 2019.

[4] X. Wang, D. Kondratyuk, E. Christiansen, K.M. Kitani, Y. Movshovitz-Attias, and E. Eban, "Wisdom of committees: an overlooked approach to faster and more accurate models," International Conference on Learning Representations (ICLR), Apr. 2022.

[5] S. Teerapittayanon, B. McDanel, and H.T. Kung, "BranchyNet: fast inference via early exiting from deep neural networks," Proceedings of the 23rd International Conference on Pattern Recognition (ICPR), pp. 2464-2469, Dec. 2016.

[6] K. Tasdemir, R. Khan, F. Siddiqui, S. Sezer, F. Kurugollu, S.B. Yengec-Tasdemir, and A. Bolat, "Advancing SQL injection detection for high-speed data centers: a novel approach using cascaded NLP," arXiv:2312.13041, Dec. 2023.

[7] W. Jitkrittum, N. Gupta, A.K. Menon, H. Narasimhan, A.S. Rawat, and S. Kumar, "When does confidence-based cascade deferral suffice?," Advances in Neural Information Processing Systems (NeurIPS), Dec. 2023.

[8] S. Hong, Y. Kaya, I.-V. Modoranu, and T. Dumitraş, "A panda? No, it's a sloth: slowdown attacks on adaptive multi-exit neural network inference," International Conference on Learning Representations (ICLR), May 2021.

[9] F. Sureda Riera, J. Bermejo Higuera, J.R. Bermejo Higuera, J.J. Martínez Herraiz, and J.A. Sicilia Montalvo, "A new multi-label dataset for web attacks CAPEC classification using machine learning techniques," Computers & Security, 120, 102788, Sep. 2022.

[10] H. Osama, O. Elebiary, Y. Qassim, M. Amgad, A. Maghawry, A. Saafan, and H. Ghalwash, "Enhanced web payload classification using WAMM: an AI-based framework for dataset refinement and model evaluation," arXiv:2512.23610, Dec. 2025.

[11] J.R. Tadhani, V. Vekariya, V. Sorathiya, S. Alshathri, and W. El-Shafai, "Securing web applications against XSS and SQLi attacks using a novel deep learning approach," Scientific Reports, 14, 1803, Jan. 2024.

[12] K. Luo, Y. Xie, S. Zhao, and J. Pan, "WADBERT: dual-channel web attack detection based on BERT models," arXiv:2601.21893, Jan. 2026.

[13] M. Bitussi and R. Doriguzzi-Corin, "X-WAD: explainable web anomaly detection," arXiv:2608.27172, Aug. 2026.

[14] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," International Conference on Learning Representations (ICLR), Apr. 2017.

[15] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," Advances in Neural Information Processing Systems (NIPS), pp. 4878-4887, Dec. 2017.

[16] H. Mozannar and D. Sontag, "Consistent estimators for learning to defer to an expert," Proceedings of the 37th International Conference on Machine Learning (ICML), pp. 7076-7087, Jul. 2020.

[17] S. Enomoto and T. Eda, "Learning to cascade: confidence calibration for improving the accuracy and computational cost of cascade inference systems," Proceedings of the AAAI Conference on Artificial Intelligence, 35(8), pp. 7331-7339, Feb. 2021.

[18] C. Guo, G. Pleiss, Y. Sun, and K.Q. Weinberger, "On calibration of modern neural networks," Proceedings of the 34th International Conference on Machine Learning (ICML), pp. 1321-1330, Aug. 2017.

[19] V.J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J.S. Gardner, I. Hubara, S. Idgunji, T.B. Jablin, J. Jiao, T.S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A.T.R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, "MLPerf inference benchmark," Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA), pp. 446-459, May 2020.

[20] S. Axelsson, "The base-rate fallacy and its implications for the difficulty of intrusion detection," Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS), pp. 1-7, Nov. 1999.

[21] C. Nadeau and Y. Bengio, "Inference for the generalization error," Machine Learning, 52(3), pp. 239-281, Sep. 2003.